UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026

INVESTIGACIÓN

MANEJO DE PARÁMETROS, PASO DE VALORES Y CLASE CALCULADORA EN JAVA



Integrantes:
Esteban Zambrano
Kevin Garcés
Cristian Gómez
Erick Cordonez
Docente: Ing. José Ruben Caiza Caizabuano

1. Introducción

En la programación orientada a objetos (POO) un programa se construye a partir de objetos que combinan datos (atributos) y acciones (métodos). Cuando un método necesita información para realizar su tarea, la recibe mediante parámetros; cuando necesita entregar un resultado, lo hace mediante un valor de retorno. Comprender cómo se declaran, cómo se llaman y cómo viajan los datos entre quien llama y el método llamado es la base para escribir código modular, reutilizable y predecible.

Este documento investiga los temas y subtemas vistos en la clase N°02: modularidad y reutilización, estructura de un método, parámetros y argumentos, múltiples parámetros, paso por valor, métodos void y con retorno, validación de entradas y la aplicación de todo ello en una clase Calculadora. La información proviene de las diapositivas de la clase y de fuentes técnicas consultadas [1]–[4]; cuando una afirmación es análisis propio y no de una fuente, se indica.

2. Objetivos

2.1. Objetivo general

Investigar y documentar los conceptos de método, parámetro, argumento, paso por valor, retorno de valores y validación de entradas en Java, y aplicarlos al análisis de la clase Calculadora.

2.2. Objetivos específicos

- Describir la modularidad y la reutilización como ventajas de organizar el código en clases y métodos.
- Identificar los componentes de la declaración de un método y su firma.
- Diferenciar parámetros de argumentos y explicar cómo deben coincidir.
- Explicar el paso por valor para datos primitivos y de referencia, y sus consecuencias.
- Distinguir los métodos void de los métodos con retorno, y analizar la división entre cero según el tipo de dato.
- Analizar críticamente la clase Calculadora y proponer mejoras.

3. Marco teórico

3.1. Modularidad, objetos, métodos y reutilización

Las diapositivas de la clase plantean cuatro ideas que sirven de punto de partida. La modularidad consiste en organizar el código en clases y métodos, de modo que un programa grande se divida en partes pequeñas y manejables. Los objetos representan elementos del mundo real mediante atributos (lo que son) y comportamientos (lo que hacen). Los métodos definen las acciones que puede realizar cada objeto. La reutilización consiste en aprovechar código ya escrito, llamándolo cuantas veces haga falta en lugar de escribirlo de nuevo.

Una calculadora ilustra bien las cuatro ideas: es un objeto cuyas acciones (sumar, restar, multiplicar, dividir) son métodos; cada operación se programa una sola vez y se reutiliza con cualquier par de números.

3.2. Anatomía de un método

Según el tutorial oficial de Java, una declaración de método tiene seis componentes, en este orden: modificadores (como public), tipo de retorno (o void si no devuelve nada), nombre, lista de parámetros entre paréntesis, lista de excepciones y cuerpo entre llaves [1]. Los únicos elementos obligatorios son el tipo de retorno, el nombre, los paréntesis y el cuerpo [1].

La firma del método (method signature) está formada por el nombre y los tipos de sus parámetros; por ejemplo, la firma de sumar(double a, double b) es sumar(double, double) [1]. Por convención, los nombres de método son verbos en minúscula, con mayúscula inicial en cada palabra posterior (calcularAreaCuadrado, mostrarMenu) [1].

Java permite la sobrecarga de métodos: varios métodos de una clase pueden compartir el nombre si sus listas de parámetros difieren en número o tipo. El compilador no considera el tipo de retorno para distinguirlos, y el tutorial recomienda usar la sobrecarga con moderación porque puede reducir la legibilidad [1].

```
public double sumar(double a, double b) {      // firma: sumar(double, double)
   return a + b;
}

public int sumar(int a, int b) {               // sobrecarga: firma sumar(int, int)
   return a + b;
}
```

3.3. Parámetros y argumentos

Los parámetros son las variables que aparecen en la declaración del método; los argumentos son los valores reales que se pasan al invocarlo [2]. Al llamar al método, los argumentos deben coincidir con los parámetros en tipo y en orden [2]. En tiempo de ejecución, cada parámetro toma el valor del argumento correspondiente [2].

Término	Qué es	Dónde aparece	Ejemplo
Parámetro	Variable declarada en la firma del método; recibe datos.	Definición del método.	double a, double b
Argumento	Valor real enviado al llamar al método.	Llamada al método.	15.5, 4.5

Un parámetro puede ser de cualquier tipo: primitivo (int, double, boolean) o de referencia (objetos y arreglos) [2]. El nombre de un parámetro debe ser único dentro de su ámbito: no

puede repetirse en el mismo método ni coincidir con una variable local del método [2]. Si un parámetro tiene el mismo nombre que un campo de la clase, lo oculta (shadowing); esto se usa sobre todo en constructores y métodos que asignan campos, pero puede dificultar la lectura [2].

3.4. Múltiples parámetros

Un método puede recibir tantos parámetros como necesite. Se separan por comas, cada uno con su tipo y nombre, y pueden ser del mismo tipo o de tipos combinados [2]. Como el emparejamiento es por posición, el orden de los argumentos importa: registrarOperacion("suma", 20.0, true) funciona, pero registrarOperacion(true, "suma", 20.0) no compila porque los tipos no coinciden en orden.

```
public void registrarOperacion(String tipoOperacion, double resultado, boolean exitosa) {
   System.out.println("Operación: " + tipoOperacion);
   System.out.println("Resultado: " + resultado);
   System.out.println("¿Exitosa?: " + exitosa);
}

registrarOperacion("suma", 20.0, true);   // argumentos en el mismo orden que los parámetros
```

Como extensión, Java ofrece varargs: si se escribe tres puntos después del tipo del último parámetro (Tipo... nombre), el método acepta cualquier cantidad de argumentos de ese tipo, y dentro del método se trata como un arreglo [2]. El método printf es un ejemplo conocido [2].

3.5. Paso por valor

En Java todos los argumentos se pasan por valor: el método recibe una copia del valor de la variable enviada [2]. Las diapositivas lo resumen así: modificar la variable del parámetro dentro del método no afecta la variable original.

3.5.1. Datos primitivos

Con tipos primitivos, los cambios hechos al parámetro existen solo dentro del método; al terminar, el parámetro desaparece y sus cambios se pierden [2]. Esto responde la pregunta de la clase: si x = 5 se pasa a un método que le suma 10 a su parámetro, x sigue valiendo 5 al volver.

```
public static void sumarDiez(int n) {
   n = n + 10;              // modifica solo la copia: n vale 15
}

int x = 5;
sumarDiez(x);
System.out.println(x);          // imprime 5
```

3.5.2. Datos de referencia

Con objetos, lo que se copia es el valor de la referencia, no el objeto. Por eso el método puede modificar los campos del objeto al que apunta la referencia (y esos cambios persisten), pero no puede hacer que la variable original apunte a otro objeto: reasignar el parámetro dentro del método no tiene efecto afuera [2].

```
class Caja { int cantidad = 5; }

public static void cambiar(Caja c) {
    c.cantidad = 20;          // modifica el objeto compartido: el cambio persiste
    c = new Caja();           // reasigna solo la copia de la referencia
    c.cantidad = 99;          // afecta al objeto nuevo, no al original
}

Caja caja = new Caja();
cambiar(caja);
System.out.println(caja.cantidad);   // imprime 20
```

Aspecto	Tipo primitivo	Tipo de referencia (objeto)
Qué se copia	El valor del dato.	El valor de la referencia (no el objeto).
¿Cambia la variable original al modificar el parámetro?	No.	No: la variable sigue apuntando al mismo objeto.
¿Se pueden modificar los campos del objeto?	No aplica.	Sí, si tienen el nivel de acceso adecuado.
Ejemplo	int x = 5;	Caja caja = new Caja();

3.6. Métodos void y métodos con retorno

Un método devuelve el control a quien lo llamó cuando termina todas sus instrucciones, cuando llega a una instrucción return o cuando lanza una excepción, lo que ocurra primero [3]. El tipo de retorno se declara en la firma y el valor se entrega con return [3].

Un método declarado void no devuelve valor y no necesita return, aunque puede usar return; (sin valor) para salir antes; intentar devolver un valor desde un método void produce un error de compilación [3]. En cambio, todo método que no es void debe contener un return con un valor cuyo tipo coincida con el declarado; por ejemplo, no se puede devolver un entero desde un método que declara boolean [3].

Característica	Método void	Método con return
¿Devuelve un valor?	No.	Sí.
Tipo de retorno en la firma	void	double, int, boolean, String, etc.
Instrucción return	Opcional, sin valor (return;).	Obligatoria, con valor del tipo declarado.
Uso típico	Imprimir, mostrar un menú, actualizar un estado.	Calcular y entregar un resultado.
Ejemplo en la calculadora	mostrarMenu()	sumar(a, b), dividir(a, b)
Cómo se invoca	calc.mostrarMenu();	double total = calc.sumar(10.5, 5.5);

3.7. Validación de entradas y división entre cero

Un método debe ser capaz de validar sus entradas, es decir, comprobar que los argumentos recibidos son aceptables antes de usarlos. El caso clásico de la calculadora es el divisor igual a cero. En Java el comportamiento depende del tipo de dato: la división entera entre cero lanza una ArithmeticException, mientras que la división de números de punto flotante (double, float) no lanza excepción y produce valores especiales: infinito si el dividendo es distinto de cero, y NaN (no es un número) si ambos son cero [4].

Operación	Tipos	Resultado	¿Lanza excepción?
10 / 0	int / int	—	Sí: ArithmeticException (/ by zero).
10.0 / 0	double / int	Infinity	No.
0.0 / 0	double / int	NaN	No.

Análisis propio: como la clase Calculadora trabaja con double, la llamada dividir(20, 0) sin validación devolvería Infinity en lugar de fallar; es decir, el programa continuaría con un valor que no es un resultado útil. La validación if (b == 0) cumple entonces una función real: detectar el caso antes de que un valor especial se propague por el resto del programa.

4. Aplicación: clase Calculadora

4.1. Estructura de la clase

La clase Calculadora de la sesión práctica combina un método void sin parámetros (mostrarMenu) con cuatro métodos con retorno y dos parámetros double cada uno. Solo dividir incluye validación.

```
public class Calculadora {
   public void mostrarMenu() {
      System.out.println("=== CALCULADORA BÁSICA EN JAVA ===");
```

```
        System.out.println("1. Sumar | 2. Restar | 3. Multiplicar | 4. Dividir");
    }

    public double sumar(double a, double b)       { return a + b; }
    public double restar(double a, double b)      { return a - b; }
    public double multiplicar(double a, double b) { return a * b; }

    public double dividir(double a, double b) {
        if (b == 0) {
            System.out.println("Error: No se puede dividir entre cero.");
            return 0;
        }
        return a / b;
    }
}
```

Método	Retorno	Parámetros	Función
mostrarMenu()	void	Ninguno	Muestra el título y las opciones.
sumar(a, b)	double	double a, double b	Devuelve a + b.
restar(a, b)	double	double a, double b	Devuelve a - b.
multiplicar(a, b)	double	double a, double b	Devuelve a * b.
dividir(a, b)	double	double a, double b	Devuelve a / b; si b es 0 informa el error y devuelve 0.

4.2. Ejecución desde la clase Main

Para usar la clase se instancia un objeto con new y se llaman sus métodos con el operador punto. Los números 10.5 y 5.5 son los argumentos; a y b, los parámetros que reciben una copia de esos valores.

```
Calculadora calc = new Calculadora();
calc.mostrarMenu();
double totalSuma = calc.sumar(10.5, 5.5);
System.out.println("Resultado Suma: " + totalSuma);
double totalDiv = calc.dividir(20, 0);   // activa la validación
```

Salida en consola:

```
=== CALCULADORA BÁSICA EN JAVA ===
1. Sumar | 2. Restar | 3. Multiplicar | 4. Dividir
```

Resultado Suma: 16.0

Error: No se puede dividir entre cero.

4.3. Análisis crítico y posibles mejoras

Las siguientes observaciones son análisis propio sobre el código de la clase:

- **Retorno de 0 ante el error.** Devolver 0 es sencillo, pero es ambiguo: 0 también es un resultado válido (por ejemplo, 0 / 5). Quien llama no puede distinguir un error de un cociente real. Alternativas: devolver Double.NaN o lanzar una excepción como IllegalArgumentException.
- **Mensaje dentro del método de cálculo.** dividir mezcla cálculo e impresión. Separar ambas tareas (el método calcula o lanza el error y quien llama decide cómo informar) favorece la modularidad y la reutilización.
- **Resultado no utilizado.** En Main, totalDiv se declara pero no se imprime; en la práctica de clase eso basta para activar la validación, pero un uso real mostraría o comprobaría el valor.
- **Comparación b == 0 con double.** Para este caso (comparar con el cero exacto) es adecuada; la comparación de dos valores double calculados por igualdad exacta, en general, requiere cuidado por la precisión del punto flotante.
- **Sobrecarga.** Se podrían añadir versiones con int, pero el tutorial recomienda usar la sobrecarga con moderación [1]; con double la calculadora ya cubre ambos casos.

5. Glosario

Término	Definición
Método	Bloque de código que realiza una tarea y se ejecuta al ser llamado.
Firma del método	Nombre del método más los tipos de sus parámetros [1].
Parámetro	Variable declarada en la firma del método que recibe un dato [2].
Argumento	Valor real enviado al método al invocarlo [2].
Paso por valor	Mecanismo por el cual el método recibe una copia del valor del argumento [2].
Referencia	Valor que identifica a un objeto; es lo que se copia al pasar un objeto a un método.
void	Tipo de retorno de un método que no devuelve ningún valor [3].
return	Instrucción que finaliza el método y entrega un valor del tipo declarado [3].
Instancia	Objeto concreto creado a partir de una clase con new.
Validación	Comprobación de que las entradas de un método son aceptables antes de usarlas.
NaN / Infinity	Valores especiales de punto flotante que resultan de operaciones como 0.0 / 0 o x / 0.0 [4].

6. Conclusiones

- La modularidad y la reutilización se logran dividiendo el código en clases y métodos; cada tarea se escribe una vez y se llama las veces necesarias.
- La firma de un método (nombre y tipos de parámetros) permite al compilador distinguirlo, y los argumentos deben coincidir con los parámetros en tipo y orden [1], [2].
- En Java todo se pasa por valor: con primitivos, el método trabaja con una copia que no altera el original; con objetos, se copia la referencia, de modo que el método puede cambiar el contenido del objeto pero no la variable original [2].
- Un método void ejecuta una acción; un método con retorno entrega un resultado cuyo tipo debe coincidir con el declarado [3].
- Validar entradas es parte del diseño de un método: con double, dividir entre cero no falla sino que produce Infinity o NaN [4], por lo que la comprobación explícita evita resultados inválidos.
- La clase Calculadora integra estos conceptos y puede mejorarse separando el cálculo de la impresión y señalando el error de forma que no se confunda con un resultado válido.

7. Referencias

[1] Oracle, «Defining Methods», The Java Tutorials. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/java/javaOO/methods.html (consultado el 21 de octubre de 2026).

[2] Oracle, «Passing Information to a Method or a Constructor», The Java Tutorials. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/java/javaOO/arguments.html (consultado el 21 de octubre de 2026).

[3] Oracle, «Returning a Value from a Method», The Java Tutorials. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/java/javaOO/returnvalue.html (consultado el 21 de octubre de 2026).

[4] Baeldung, «Division by Zero in Java». [En línea]. Disponible en: https://www.baeldung.com/java-division-by-zero (consultado el 21 de octubre de 2026).